

Manual Técnico & Arquitetura de Software

Análise Detalhada do Sistema de Gestão de Pedidos para Lanchonete (Flask, MySQL e JavaScript)

1. Visão Geral do Sistema

Este documento apresenta uma análise técnica profunda do **Sistema de Gestão de Pedidos**. O ecossistema foi projetado seguindo o padrão arquitetural de separação de responsabilidades, combinando a robustez do backend em Python com o microframework Flask, persistência de dados relacional com o MySQL, e uma interface rica, reativa e assíncrona orientada por JavaScript no frontend.

A Analogia da Lanchonete Real

Para compreender perfeitamente o fluxo de dados do projeto, imagine o ecossistema dividido em três papéis:

- **O Banco de Dados (MySQL):** É o grande livro de receitas e o caderno de fiados fixo no caixa. Nada se perde aqui.
- **O Servidor Backend (Flask/Python):** É o balcão central e o Gerente da Cozinha. Ele recebe os pedidos da mesa, confere as regras de negócio e dita o que entra e sai.
- **O Frontend (HTML/CSS/JS):** É o garçom e o painel de LED eletrônico das mesas. Ele exibe os dados de forma bonita para o cliente e avisa o gerente instantaneamente sobre cliques e interações.

2. Arquitetura e Fluxo de Dados (Engrenagens do Sistema)

O projeto é estruturado de forma que o cliente nunca converse diretamente com o banco de dados por motivos de segurança e performance. O fluxo de uma requisição segue estritamente os passos abaixo:

1. O usuário interage com a interface (ex: clica em "Deletar" na tela `listar.html`).
2. O código **JavaScript** captura o evento, realiza uma animação visual para melhorar a experiência do usuário e dispara uma chamada assíncrona (`fetch API`) para o backend.
3. O **Flask (app.py)** intercepta a requisição na rota correspondente (ex: `/pedidos/<id>` com o método HTTP `DELETE`).
4. O controller do Flask aciona a camada de modelo (**Pedido no arquivo pedido.py**), que por sua vez solicita uma conexão ativa com o banco através da classe **Database (database.py)**.
5. O comando SQL estruturado é executado no servidor **MySQL**. O banco processa e devolve a confirmação de linhas afetadas.
6. A resposta faz o caminho inverso, sendo empacotada em formato JSON pelo Flask e devolvida ao JavaScript, que exibe um alerta de sucesso na tela sem a necessidade de recarregar a página inteira.

3. O Coração dos Dados: Modelagem do Banco (MySQL)

A persistência é mantida através de um banco de dados relacional chamado `lanchonete`. A tabela centralizada armazena os atributos fundamentais de um ciclo de venda alimentícia.

Estrutura da Tabela de Pedidos

Nome do Campo	Tipo de Dado	Restrições / Atributos	Descrição Opcional
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único gerado automaticamente para cada pedido.
cliente	VARCHAR(100)	NOT NULL	Nome completo ou identificação do cliente que realizou a compra.
lanche	VARCHAR(100)	NOT NULL	Nome ou descrição do item/produto alimentício solicitado.
quantidade	INT	NOT NULL	Volume de unidades adquiridas (deve ser maior ou igual a 1).
status	VARCHAR(50)	NOT NULL, DEFAULT 'Pendente'	Estado atual do pedido na linha de produção (Pendente, Preparando, Pronto, Entregue).

```
-- Script de criação estrutural e carga inicial de amostragem
CREATE DATABASE IF NOT EXISTS lanchonete;
USE lanchonete;

CREATE TABLE IF NOT EXISTS pedidos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  cliente VARCHAR(100) NOT NULL,
  lanche VARCHAR(100) NOT NULL,
  quantidade INT NOT NULL,
  status VARCHAR(50) NOT NULL DEFAULT 'Pendente'
);
```

4. O Motor Backend: Infraestrutura Python & Flask

O backend está subdividido de forma modular e inteligente, isolando regras de banco de dados das lógicas de rotas visuais. Essa separação limpa facilita manutenções futuras (como trocar o banco MySQL por PostgreSQL sem quebrar as telas do usuário).

4.1 Classe Database (database.py)

Responsável pelo gerenciamento do ciclo de vida das conexões. Implementa de forma robusta blocos `try/except/finally`, garantindo que as conexões abertas com o servidor MySQL sejam obrigatoriamente fechadas ao término de cada transação, prevenindo o vazamento de memória ou estouro do limite de conexões simultâneas do banco de dados (Connection Pooling manual).

- `execute_query()`: Abre conexão, prepara e executa comandos de alteração estrutural (INSERT, UPDATE, DELETE) e realiza o `commit()` para persistir as alterações na tabela.
- `fetch_all()`: Executa consultas de leitura (SELECT), convertendo automaticamente as tuplas retornadas pelo MySQL em um dicionário Python estruturado (`dictionary=True`), o que simplifica a conversão nativa para objetos JSON legíveis no frontend.

4.2 Classe de Modelo Pedido (pedido.py)

Encapsula as consultas SQL cruas do projeto. Ela funciona como uma camada de abstração (DAO - Data Access Object), onde o arquivo principal do Flask apenas invoca funções limpas orientadas a objetos, ocultando os comandos `SELECT`, `INSERT` ou `UPDATE` internos.

4.3 Controlador de Rotas Principal (app.py)

O maestro da aplicação. É ele quem escuta as requisições web do navegador nas portas do computador. Ele expõe dois tipos principais de rotas:

Diferença de Rotas no Flask

Rotas Visuais (Páginas HTML): Retornam telas prontas renderizadas pelo motor Jinja2 do Flask através da função `render_template()`.

Rotas de API (Endpoints JSON): Não possuem interface gráfica direta. Retornam dados estruturados em formato texto JSON através do `jsonify()` e servem para alimentar os scripts assíncronos do JavaScript.

5. A Camada de Apresentação Dinâmica: JavaScript Frontend

As páginas da lanchonete utilizam JavaScript moderno com chamadas assíncronas assinaladas pelas diretivas `async/await`. Isso promove um comportamento fluido onde os dados são salvos, editados e atualizados sem que o usuário perceba lentidão ou piscadas de recarregamento de tela.

Análise Funcional dos Scripts

- **Gerenciamento de Estado Seguro:** Implementação de flags de controle como a variável `isSubmitting` na tela de cadastro e edição. Essa flag atua como uma trava lógica. Assim que o usuário clica em enviar, o botão é congelado. Isso impede disparos duplos acidentais no banco de dados caso o cliente clique repetidamente no botão por ansiedade ou lentidão da rede.

- **Filtro em Tempo Real por Input:** O arquivo `listar.html` escuta o evento `input` do teclado na barra de pesquisas. Ele não refaz chamadas pesadas ao servidor a cada letra digitada; em vez disso, ele filtra a matriz local de dados previamente carregada na memória do navegador (`todosOsPedidos.filter()`), entregando uma experiência de busca instantânea de milissegundos.
- **Sanitização e Proteção contra Ataques XSS:** Inclusão sistemática da função utilitária `escapeHtml()`. Antes de injetar dados de strings vindos do banco de dados direto no HTML dinâmico, os caracteres sensíveis como `<`, `>` ou aspas são convertidos em entidades de texto inofensivas. Isso blinda a aplicação contra falhas graves de Cross-Site Scripting (injeção de códigos JavaScript maliciosos por usuários mal-intencionados através do formulário de nomes de clientes).

6. Resumo das Rotas da API e Seus Métodos HTTP

Endpoint da Rota	Método HTTP	Ação Executada	Resposta Esperada (Sucesso)
<code>/api/pedidos</code>	GET	Recupera a lista completa de pedidos cadastrados do banco em ordem inversa.	Array de objetos JSON com todos os registros.
<code>/api/pedidos</code>	POST	Cria um novo registro de pedido validando os campos obrigatórios enviados.	Status 201 - Confirmação de inserção bem-sucedida.
<code>/api/pedidos/<id></code>	GET	Busca os dados individualizados de um único pedido pelo número identificador.	Objeto JSON único com as propriedades daquele ID.
<code>/api/pedidos/<id></code>	PUT	Modifica as informações ou altera o status de produção do pedido específico.	Status 200 - Mensagem de atualização concluída.
<code>/api/pedidos/<id></code>	DELETE	Remove de forma permanente o registro selecionado da base de dados.	Status 200 - Alerta de remoção permanente com sucesso.

Conclusão sobre o Padrão de Engenharia Empregado

O projeto destaca-se pela excelente aplicação de arquitetura monolítica leve com desacoplamento de persistência de dados. A aplicação consome poucos recursos do servidor e responde com tempos de latência reduzidos devido às interações assíncronas do frontend, qualificando-se como um modelo ideal, escalável e seguro para gestão de fluxos de atendimento comerciais rápidos.